

Законы масштабирования, Тщательно

Дата: 24 июня 2026 г. | Предполагаемое время чтения: 25 мин. | Автор: Лилиан Венг

Содержание

Законы масштабирования - одно из наиболее важных эмпирических открытий в области глубокого обучения. Наблюдение простое по форме: потери на обучение L предсказуемо уменьшаются по мере увеличения размера модели N , размера набора данных D и вычисления C , следуя степенной кривой, которая отображается в виде прямой линии на логарифмическом графике. Мы можем рассматривать законы масштабирования как основу для описания взаимосвязи между вычислениями, потерями, размером модели и данными; по сути, речь идет о том, как оптимально распределить ценные вычислительные ресурсы между N и D .

Такая предсказуемость делает законы масштабирования весьма ценными на практике. Обычный рабочий процесс заключается в подгонке законов масштабирования к нескольким небольшим прогонам, а затем в экстраполяции для оценки токена и вычисления требований для более крупных моделей.

Символ	Примечание
N	Размер модели, измеряемый количеством параметров.
D	Размер обучающего набора данных, обычно измеряемый в количестве токенов.
C	Обучающие вычисления во флопах. В качестве полезного приближения, C приблизительно $6ND$ (Kaplan et al. 2020), где $2ND$ приходится на прямой проход и $4ND$ на обратное распространение.
E	Невосполнимая потеря
$L, \hat{L}(\cdot)$	Потеря теста / функция прогнозирования потери теста; также может относиться к потере обучения, поскольку они сильно коррелируют.
ϵ	Ошибка обобщения.

Первые дни: предсказуемость потерь МЛ

Предсказуемость ошибки обобщения в зависимости от масштаба уже исследовалась до того, как законы масштабирования стали общепринятой концепцией.

Амари и др. (1992) вывели четыре типа кривых обучения, используя байесовский подход и отожденное приближение.

1. Детерминированный алгоритм обучения, безошибочные данные, одно уникальное решение: $\epsilon \sim c \cdot D^{-1}$, где c — некоторая константа.
2. Детерминированный алгоритм обучения, данные без ошибок, несколько эквивалентных решений: $\epsilon \sim c \cdot D^{-2}$; с каждой новой точкой данных обучение происходит быстрее, поскольку модель изучает только оптимальное многообразие параметров, а не находит единственную точку решения.
3. Детерминированный алгоритм обучения, зашумленные данные: $\epsilon \sim c \cdot D^{-1/2}$; шумы в данных затрудняют обучение.
4. Алгоритм стохастического обучения, зашумленные данные: $\epsilon \sim c \cdot D^{-1/2} + E$; здесь неприводимая потеря E — это остаточная ошибка, которую стохастический обучающийся алгоритм не может уменьшить, например, когда модель исчерпывает



возможности для обработки больших данных. Все четыре типа кривых обучения подчиняются степенному закону:

$$\epsilon(m) \propto D^\alpha + E$$

где E может быть равно 0, а $\alpha = -2, -1, -1/2$. Несмотря на то, что их теоретическая модель основана на упрощенной задаче бинарной классификации, она указывает на перспективное направление для создания эмпирических моделей прогнозирования потерь в машинном обучении.

Одно из самых ранних эмпирических исследований, проведенных Хестнессом и др. (2017), объяснило взаимосвязь между ошибкой обобщения, размером модели и данными. Для заданного размера обучающих данных они определили наиболее подходящий размер модели с помощью поиска по сетке, а затем построили график потерь в зависимости от размера обучающего набора данных. В четырех различных областях глубокого обучения (нейромашинный перевод, классификация изображений, языковое моделирование и распознавание речи) наблюдалась повторяющаяся закономерность, при которой:

- Ошибка обобщения масштабируется в виде степенного закона для ряда факторов (например, размера данных).
- Усовершенствования модели сдвигают кривую ошибки, но, похоже, не влияют на степенной показатель.
- Интересно, что архитектура изменяет смещение (E) степенного соответствия, но не меняет показатель степени (α). Наклон степенного закона, по-видимому, является свойством проблемной области, а не архитектуры модели.
- Количество параметров модели N , необходимых для соответствия набору данных размера D , также масштабируется по степенному закону.

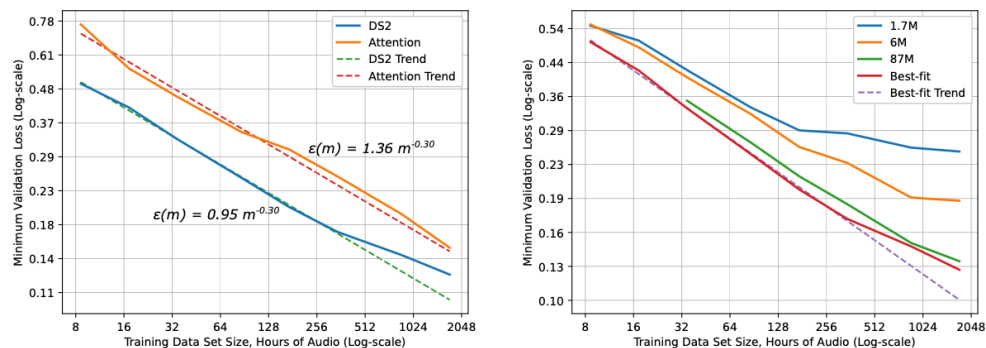


Figure 1: Кривые обучения для (Слева) модели Deep-Speech-2 (DS2) и attention speech model, а также для (Справа) моделей DS2 различных размеров. Потери небольших моделей достигают максимума, когда обучающие данные становятся большими. (Источник изображения: Хестнесс и др., 2017)

Концептуальная иллюстрация разбивает кривую обучения на три этапа. В области с небольшим объемом данных, когда обучающих сигналов недостаточно, модель работает лишь ненамного лучше, чем случайное угадывание. В середине ("область степенного распределения") мы наблюдаем степенную зависимость между потерями, данными и размером модели. Конечная область неустраняемых ошибок может быть отнесена к таким факторам, как шум в данных.

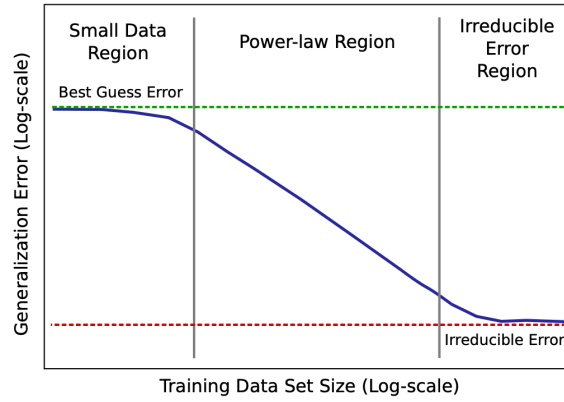


Figure 2: Иллюстрация фаз степенной кривой обучения. (Источник изображения: Hestness et al. 2017)

Розенфельд и др. (2020) пошли дальше и попытались смоделировать ошибку как совместную функцию от размера модели. N и размер данных D , в различных архитектурах (ResNet, WRN, LSTM, Transformer) и оптимизаторах (Adam, варианты SGD). Эмпирически они обнаружили, что при фиксировании одной оси ошибка уменьшается по степенному закону в зависимости от другой:

$$\hat{L}(D, N) \approx \frac{A}{N^\alpha} + E_N, \quad \hat{L}(D, N) \approx \frac{B}{D^\beta} + E_D$$

которые можно объединить в одну форму:

$$\hat{L}(D, N) \approx \frac{A}{N^\alpha} + \frac{B}{D^\beta} + E$$

где $A > 0, B > 0, \alpha \geq 0, \beta \geq 0$ являются скалярными константами и E не зависит ни от того, ни от другого N или D .

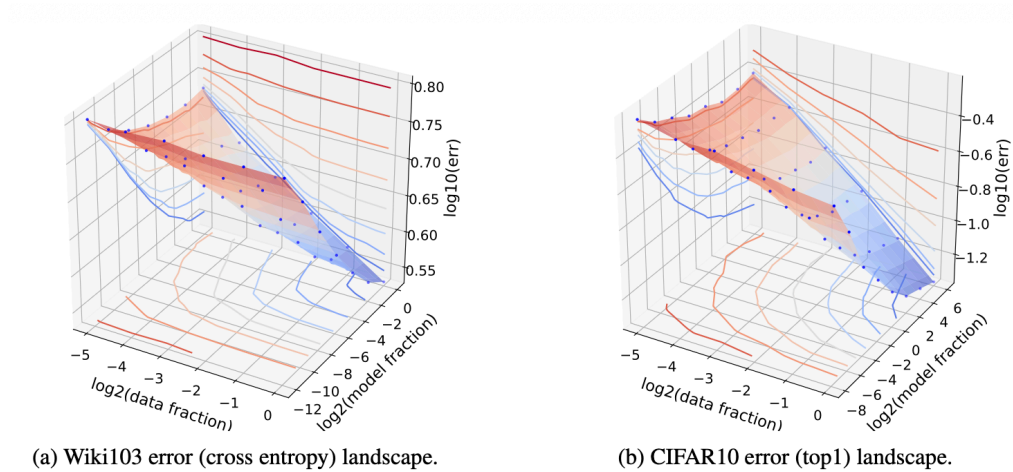


Figure 3: Трехмерный контурный график зависимости размера данных, размера модели и ошибки обобщения в логарифмическом масштабе.

Синие точки получены в результате эмпирических экспериментов, а поверхность представляет собой линейную интерполяцию между синими точками. (Источник изображения: Rosenfeld et al. 2020)

Таким образом, они могут построить модель прогнозирования в виде простой параметрической функции с $\theta = \langle A, B, E, \alpha, \beta \rangle$ для прогнозирования ожидаемых убытков $(D, N) >$ определенных пороговых значений, обучаясь только на наборе небольших обучающих конфигураций, $(D, N) <$ определенные пороговые значения.

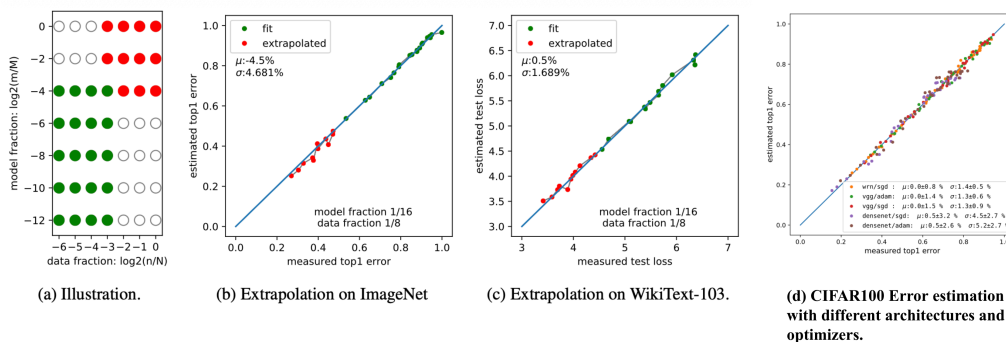


Figure 4: Подгонка параметрической модели ошибок к мелкомасштабным конфигурациям и экстраполяция на более крупные режимы модели / данных: (a) Иллюстрация установки эксперимента; Результаты эксперимента на (b) ImageNet, (c) WikiText-103 и (d) оценка ошибок CIFAR100 с использованием трех архитектур (WRN, VGG, DenseNet) и двух оптимизаторов (SGD, Adam). (Источник изображения: Розенфельд и др., 2020)

Примечание: в этих ранних работах используется классическая теория обучения, основанная на таких понятиях, как размерность VC (мощность наибольшего множества точек, которое может разделить модель), в качестве косвенного показателя способности к обобщению, но в современных исследованиях в области глубокого обучения размерность VC часто оказывается слишком грубой мерой для объяснения поведения модели, а эмпирические степенные законы оказались гораздо более точными и практичными, чем наихудшие границы, которые дает теория.

Законы масштабирования в бесконечной области данных

Законы масштабирования Каплана и др.

Каплан и др. (2020) популяризировали концепцию законов масштабирования в сообществе языкового моделирования. Они обнаружили, что потеря кросс-энтропийного теста L масштабируется в степенном порядке для каждого размера модели N (без учета слоев внедрения), размер набора данных D , и обучающие вычисления C на много порядков. Полученные результаты согласуются с ранними исследованиями, описанными в предыдущем разделе, но Каплан и соавторы формализовали эту концепцию, сосредоточившись на языковых моделях Transformer, и провели эмпирические эксперименты в более широком масштабе, с размером модели от 768 млн до 1,5 млрд параметров без эмбедингов и размером набора данных от 22 млн до 23 млрд токенов. Во всех обучающих прогонах, описанных в статье, использовался график скорости обучения с линейным прогревом в течение 3000 шагов, за которым следовал косинусный спад до нуля.

Список ключевых выводов:

- Потеря L масштабируется как степенной закон с N , D , и C по отдельности; для оптимальной производительности все три параметра должны масштабироваться одновременно.
- Кривые обучения подчиняются предсказуемым степенным законам, параметры которых практически не зависят от размера модели.
- Более крупные модели более эффективны с точки зрения выборки, то есть они достигают заданного уровня потерь за меньшее количество шагов оптимизации и с использованием меньшего количества точек данных, чем небольшие модели.
- Архитектурные детали (ширина, соотношение сторон и т. д.) имеют меньшее значение, чем масштаб.
- Потери при обучении и потери при тестировании положительно коррелируют. (Звучит банально, но это основа для претренинга. С другой стороны, вопрос о том, влияет ли

- улучшение показателей на этапе предварительного обучения на оценку после обучения, требует отдельного исследования.)
- При фиксированном бюджете вычислительных ресурсов эффективнее обучить очень большую модель и остановиться до достижения сходимости, чем обучать модель меньшего размера до полной сходимости. Именно в этом вопросе законы масштабирования Chinchilla (см. следующий раздел) расходятся с результатами: Kaplan et al. переоценили оптимальный размер модели, поскольку подобранный ими показатель был больше.

Они обобщают совместную зависимость от N и D в одном уравнении:

$$\hat{L}(N, D) = \left[\left(\frac{a}{N} \right)^{\frac{\alpha}{\beta}} + \frac{b}{D} \right]^{\beta}$$

Приятным следствием этой формы является то, что степень переоснащения (т. е. Модель сложная или данных мало) зависит преимущественно от соотношения $N^{\alpha/\beta}/D$, что указывает на то, что объем данных должен увеличиваться в определенной пропорции к росту размера модели, чтобы избежать ограничения данных при обучении.

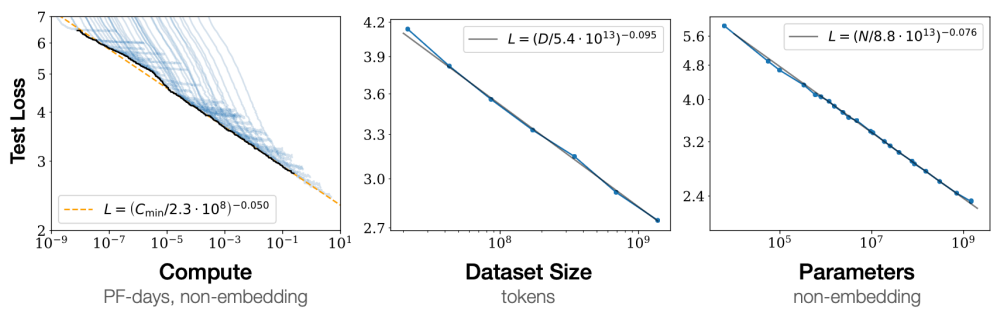


Figure 5: Тестируйте потери в виде степенного закона в вычислениях, размере набора данных и параметрах, охватывающих многие порядки величины. (Источник изображения: Kaplan et al., 2020)

Наиболее влиятельным и, оглядываясь назад, наиболее оспариваемым выводом было оптимальное для вычислений распределение ресурсов. Kaplan et al. нашел $N_{\text{opt}} \propto C^{0.73}$ и пришли к выводу, что размер модели должен расти быстрее, чем размер набора данных. В частности, при увеличении вычислительных мощностей в 10 раз они предложили увеличить размер модели примерно в 5,5 раза, а количество обучающих токенов — всего в 1,8 раза. В статье Chinchilla эта рекомендация была опровергнута на том основании, что из-за неё большие модели остаются сильно недообученными.

Другой полезный анализ, проведенный Капланом и др., приблизительно определяет количество необходимых тренировочных провалов на основе D и N . Каждое умножение-сложение засчитывается как ~ 2 провала.

Operation	Parameters	FLOPs per Token
Embed	$(n_{\text{vocab}} + n_{\text{ctx}}) d_{\text{model}}$	$4d_{\text{model}}$
Attention: QKV	$n_{\text{layer}} d_{\text{model}} 3d_{\text{attn}}$	$2n_{\text{layer}} d_{\text{model}} 3d_{\text{attn}}$
Attention: Mask	—	$2n_{\text{layer}} n_{\text{ctx}} d_{\text{attn}}$
Attention: Project	$n_{\text{layer}} d_{\text{attn}} d_{\text{model}}$	$2n_{\text{layer}} d_{\text{attn}} d_{\text{embd}}$
Feedforward	$n_{\text{layer}} 2d_{\text{model}} d_{\text{ff}}$	$2n_{\text{layer}} 2d_{\text{model}} d_{\text{ff}}$
De-embed	—	$2d_{\text{model}} n_{\text{vocab}}$
Total (Non-Embedding)	$N = 2d_{\text{model}} n_{\text{layer}} (2d_{\text{attn}} + d_{\text{ff}})$	$C_{\text{forward}} = 2N + 2n_{\text{layer}} n_{\text{ctx}} d_{\text{attn}}$

Figure 6: Параметр и вычислительная оценка для различных компонентов архитектуры Трансформатора с учетом количества слоев n_{layer} , ширина модели d_{model} ($= d_{\text{embed}}$; обозначения в исходной таблице противоречивы), размер слоя прямой связи d_{ff} (часто эквивалентно $4d_{\text{model}}$, измерение внимания d_{attn} (часто эквивалентно d_{model}), длина контекста n_{ctx} и размер словарного запаса n_{vocab} . (Источник изображения: Kaplan et al. 2020)

Учитывая стандартную конфигурацию, где $d_{\text{attn}} = d_{\text{model}} = d_{\text{ff}}/4$ и исключение слоев внедрения из N и прямое вычисление для каждого токена:

$$\begin{aligned} N &= n_{\text{layer}} d_{\text{model}} 3d_{\text{attn}} + n_{\text{layer}} d_{\text{attn}} d_{\text{model}} + n_{\text{layer}} 2d_{\text{model}} d_{\text{ff}} && \text{; no embedding layer} \\ &= 2 n_{\text{layer}} d_{\text{model}} (2d_{\text{attn}} + d_{\text{ff}}) \\ &= 12 n_{\text{layer}} d_{\text{model}}^2 \end{aligned}$$

$$\begin{aligned} C_{\text{fwd}} &= 2n_{\text{layer}} (d_{\text{model}} 3d_{\text{attn}} + n_{\text{ctx}} d_{\text{attn}} + d_{\text{attn}} d_{\text{embed}} + 2d_{\text{model}} d_{\text{ff}}) \\ &= 2n_{\text{layer}} (12d_{\text{model}}^2 + n_{\text{ctx}} d_{\text{attn}}) \\ &= 2N + 2n_{\text{layer}} n_{\text{ctx}} d_{\text{attn}} \\ &\approx 2N && \text{; assuming } n_{\text{ctx}} < 12d_{\text{model}} \text{ and the } n_{\text{ctx}} \text{ term is relatively small.} \end{aligned}$$

Затем мы подсчитываем провалы обратного прохода как удвоенные провалы прямого прохода, потому что обратное распространение выполняет два матричных умножения для градиентов относительно входных активаций и весов соответственно. Таким образом, в общей сложности обучающие флопы на один токен составляют приблизительно $6N$, и общее количество провалов за тренировку в течение D токены - это $C \approx 6ND$.

Законы масштабирования шиншиллы

В статье Chinchilla (Hoffmann et al. 2022) изучалась взаимосвязь между оптимальным размером модели и N (общее количество параметров, включая эмбединги) и количество токенов D при фиксированном бюджете на вычислительные ресурсы C с более тщательным планированием эксперимента и пришли к несколько иному ответу, чем Каплан и др..



Figure 7: Вы должны знать, как выглядит шиншилла 😊 (Источник изображения: сгенерировано ChatGPT)

Центральный вопрос заключается в наилучшей стратегии распределения ресурсов с учетом ограничений $\text{FLOPs}(N, D) = C \approx 6ND$. Другими словами, когда у нас есть только ограниченные провалы (заданное количество графических процессоров, работающих в течение заданного периода времени), как мы должны выбирать между большим количеством токенов данных и большим количеством параметров модели?

$$N_{\text{opt}}(C), D_{\text{opt}}(C) = \arg \min_{\text{s.t. } \text{FLOPs}(N, D) = C} \hat{L}(N, D)$$

В статье Chinchilla представлены три тщательно продуманных метода аппроксимации законов масштабирования.

В ходе эмпирических экспериментов было отсканировано более 400 моделей с размерами от 70 м до более чем 16 Б параметров и обучающими токенами от 5 Б до 500 Б. Эксперименты проводились в предположении, что каждый обучающий токен уникален (режим бесконечных данных). Во всех запусках использовался косинусоидальный график скорости обучения, уменьшающийся в 10 раз за период обучения. Широкий выбор размеров модели определяет границу, оптимальную для вычислений.

Метод 1: фиксируйте размеры модели, варьируйте бюджет токенов



Для каждого количества параметров N , подготовьте несколько ранов с разными бюджетами токенов и запишите минимальный убыток, достигнутый за бюджет ФЛОПА. C .

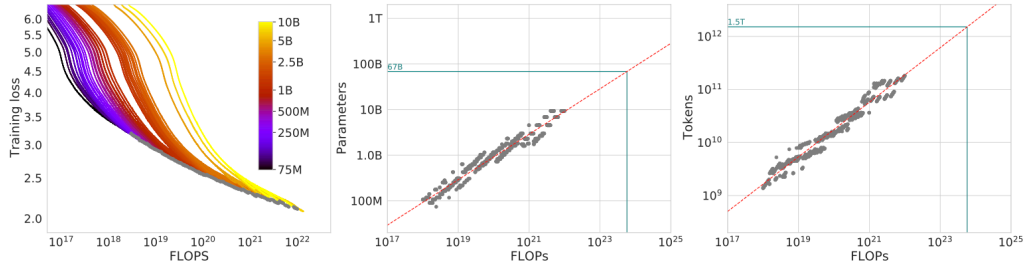


Figure 8: Метод шиншиллы 1: тренируем кривые потерь по бюджетам флопов для анализа размеров моделей. (Источник изображения: Hoffmann et al., 2022)

Метод 2: профили IsoFLOP

Исправьте бюджет вычислений C и построим график зависимости окончательных потерь от количества параметров N . Каждая кривая iso-FLOP представляет собой примерно параболу в логарифмическом пространстве, и ее минимум указывает на оптимальный размер модели для данного вычислительного бюджета. Затем повторение по бюджетам прослеживает степенную линию на графике.

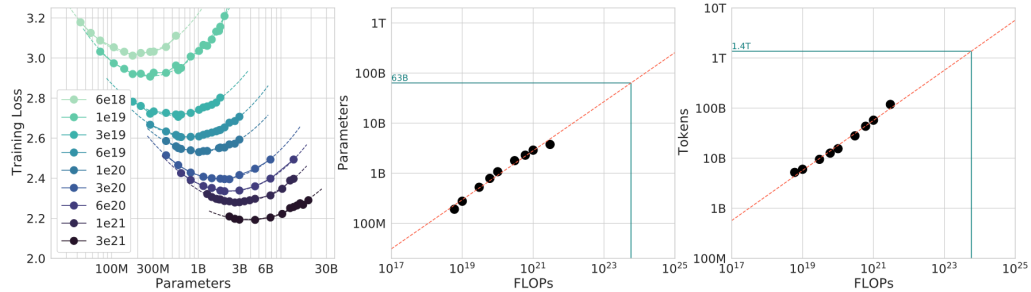


Figure 9: Метод «шиншилла» 2: параболы IsoFLOP; минимум каждой кривой — это оптимальный с точки зрения вычислительных ресурсов размер модели для данного бюджета. (Источник изображения: Hoffmann et al. 2022)

Метод 3: Параметрическая аппроксимация

Непосредственная аппроксимация той же параметрической функцией, что и в [Rosenfeld et al. \(2020\)](#)

$$\hat{L}(N, D) = \frac{A}{N^\alpha} + \frac{B}{D^\beta} + E$$

На самом деле мы можем получить приближенное значение оптимального решения в замкнутой форме $N_{\text{opt}}(C)$, $D_{\text{opt}}(C)$ путем минимизации $\hat{L}(N, D)$ в условиях ограничения $\text{FLOPs}(N, D) = C \approx 6ND$.

Сначала давайте сократим выражение, оставив только N :

$$\begin{aligned}\hat{L}(N) &= AN^{-\alpha} + B\left(\frac{C}{6}\right)^{-\beta} N^{\beta} + E \\ \hat{L}'(N) &= -\alpha AN^{-\alpha-1} + \beta B\left(\frac{C}{6}\right)^{-\beta} N^{\beta-1} = 0 \quad ; \text{ derivative wrt } N \text{ should be zero.} \\ \text{Thus } \alpha AN^{-\alpha-1} &= \beta B\left(\frac{C}{6}\right)^{-\beta} N^{\beta-1} \\ \alpha A &= \beta B\left(\frac{C}{6}\right)^{-\beta} N^{\alpha+\beta} \\ N_{\text{opt}} &= \left(\frac{\alpha A}{\beta B}\right)^{\frac{1}{\alpha+\beta}} \left(\frac{C}{6}\right)^{\frac{\beta}{\alpha+\beta}} \\ D_{\text{opt}} &= \frac{C}{6N_{\text{opt}}} = \left(\frac{\beta B}{\alpha A}\right)^{\frac{1}{\alpha+\beta}} \left(\frac{C}{6}\right)^{\frac{\alpha}{\alpha+\beta}}\end{aligned}$$

Когда $\alpha \approx \beta$, размер модели и обучающие токены должны масштабироваться с одинаковой скоростью.

Чтобы найти оптимальный $\theta = \langle A, B, E, \alpha, \beta \rangle$ В статье Chinchilla используется функция потерь Хьюбера (устойчивая к выбросам; $\delta = 10^{-3}$) и алгоритм L-BFGS (хорошо подходит для подбора кривой с небольшим количеством параметров).

$$\begin{aligned}\min_{A, B, E, \alpha, \beta} \sum_{\text{runs } \{i\}} \text{Huber}_{\delta}(\log \hat{L}(N_i, D_i) - \log L_i) \\ \text{where } \text{Huber}_{\delta}(x) = \begin{cases} \frac{1}{2}x^2 & \text{for } |x| \leq \delta \\ \delta \cdot (|x| - \frac{1}{2}\delta), & \text{otherwise.} \end{cases}\end{aligned}$$

Chinchilla приходит к ответу с помощью трех взаимодополняющих методов, конечные результаты которых совпадают, и это одна из причин, почему результат получился весьма убедительным.

Approach	Coeff. a where $N_{\text{opt}} \propto C^a$	Coeff. b where $D_{\text{opt}} \propto C^b$
1. Minimum over training curves	0.50 (0.488, 0.502)	0.50 (0.501, 0.512)
2. IsoFLOP profiles	0.49 (0.462, 0.534)	0.51 (0.483, 0.529)
3. Parametric modelling of the loss	0.46 (0.454, 0.455)	0.54 (0.542, 0.543)
Kaplan et al. (2020)	0.73	0.27

Figure 10: Эти три метода согласуются с оптимальной для вычислений границей, где $N_{\text{opt}} \propto C^{0.5}$, но не согласны с Капланом и соавторами. Обратите внимание, что результаты третьего метода немного отличаются от результатов двух других, что мы объясним позже. (Источник изображения: Hoffmann et al. 2022)

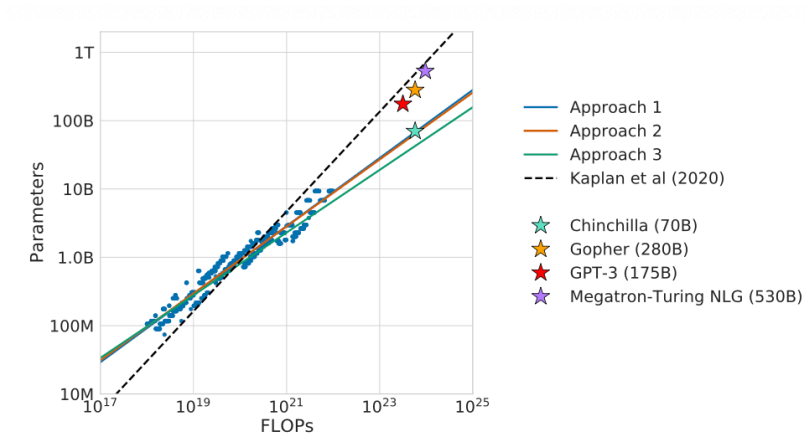


Figure 11: График предсказаний шиншиллы с помощью трех различных подходов, а также предсказаний Каплана и др. (2020). Все три метода предполагают, что несколько основных магистров права в то время были недостаточно подготовлены. (Источник изображения: Hoffmann et al., 2022)

Утверждение в статье о Chinchilla о том, что большинство больших моделей (на тот момент, примерно в 2022 году) были недостаточно обучены, подтверждается известной

демонстрацией: при том же вычислительном бюджете, что и у Gopher (Рэй и др., 2021; 280 млрд параметров, 300 млрд токенов), они обучили Chinchilla (70 млрд параметров, 1,4 трлн токенов) — модель в 4 раза меньше, но обученная примерно на 4 раза большему количеству токенов, — и она превзошла Gopher по всем показателям.

Согласование результатов Каплана и Шиншиллы

Законы масштабирования Шиншиллы расходятся с выводами Каплана и др. следующим образом:

- Вместо “наращивайте модель быстрее, чем данные” ($N_{\text{opt}} \propto C^{0.73}$), при каждом удвоении размера модели следует также удваивать количество обучающих токенов ($N_{\text{opt}} \propto C^{0.5}$).
- Вместо того чтобы “обучать большую модель и останавливаться перед конвергенцией”, вам следует обучать меньшую модель на большем количестве данных.

Обе статьи по-прежнему придерживаются одного и того же основополагающего принципа, но они расходятся во мнениях относительно того, в чем заключается оптимальный компромисс между размером и токеном. Почему они так сильно расходятся во мнениях?

Отличие 1: Каплан и соавторы проводили эксперименты в основном с небольшими моделями. Каплан и соавторы проводили эксперименты в основном с небольшими моделями, в то время как эксперименты, описанные в статье о Chinchilla, охватывали более чем в 10 раз большие масштабы. При экстраполяции в логарифмическом пространстве небольшая разница в аппроксимации может привести к большим различиям (см. [простейшую симуляцию](#)).

Разница 2: количество параметров встраивания имеет значение для небольших моделей. В режиме с небольшим количеством параметров параметры встраивания составляют немалую долю от общего числа, поэтому их подсчет имеет значение. Пирс и Сонг (2024) провели тщательный анализ в этом направлении. Давайте воспользуемся $N_{\setminus E}$, $C_{\setminus E}$ для обозначения размера модели и вычислений без использования встраивания N , C для подсчета общего количества параметров.

- Kaplan et al.: $N_{\setminus E}^* \propto C_{\setminus E}^{0.73}$ (без встраивания)
- Шиншилла: $N^* \propto C^{0.50}$ (всего)

Чтобы преодолеть это, они устанавливают взаимосвязь между общими параметрами N_T и не встраиваемые параметры $N_{\setminus E}$, для некоторой постоянной ω :

$$N = N_{\setminus E} + \omega N_{\setminus E}^{1/3}.$$

Эта форма обладает такими приятными свойствами, как строго возрастающая функция и

$$\lim_{N \rightarrow \infty} \frac{N}{N_{\setminus E}} = 1 + \omega N_{\setminus E}^{-2/3}, \lim_{N_{\setminus E} \rightarrow \infty} \frac{N}{N_{\setminus E}} = 1.$$

Подключаем это к уравнению законов шиншиллы,

$$\begin{aligned} L(N_{\setminus E}, C_{\setminus E}) &= A(N_{\setminus E} + \omega N_{\setminus E}^{1/3})^{-\alpha} + B\left(\frac{C_{\setminus E}}{6}\right)^{-\beta} N_{\setminus E}^{\beta} + E \\ L'(N_{\setminus E}, C_{\setminus E}) &= -\alpha A(N_{\setminus E} + \omega N_{\setminus E}^{1/3})^{-\alpha-1} \left(1 + \frac{\omega}{3} N_{\setminus E}^{-2/3}\right) + \beta B\left(\frac{C_{\setminus E}}{6}\right)^{-\beta} N_{\setminus E}^{\beta-1} = 0 \quad ; \text{ derivative wrt } N \\ \text{Rearrange to get } &\alpha A(N_{\setminus E}^* + \omega N_{\setminus E}^{*1/3})^{-\alpha-1} \left(1 + \frac{\omega}{3} N_{\setminus E}^{*-2/3}\right) = \beta B\left(\frac{C_{\setminus E}}{6}\right)^{-\beta} N_{\setminus E}^{*\beta-1} \\ &6^{-\beta} \frac{\alpha A}{\beta B} (N_{\setminus E}^* + \omega N_{\setminus E}^{*1/3})^{-\alpha-1} \left(1 + \frac{\omega}{3} N_{\setminus E}^{*-2/3}\right) N_{\setminus E}^{*1-\beta} = C_{\setminus E}^{-\beta} \\ &6\left(\frac{\beta B}{\alpha A}\right)^{\frac{1}{\beta}} (N_{\setminus E}^* + \omega N_{\setminus E}^{*1/3})^{\frac{1+\alpha}{\beta}} (N_{\setminus E}^* + \frac{\omega}{3} N_{\setminus E}^{*1/3})^{-\frac{1}{\beta}} N_{\setminus E}^* = C_{\setminus E} \end{aligned}$$

Взаимосвязь между $C_{\setminus E}$ и $N_{\setminus E}$ В приведенном выше уравнении уже нет чистого степенного закона. Мы можем лишь приблизительно аппроксимировать его локально как $N_{\setminus E}^* \propto C_{\setminus E}^g$,

где g является локальным показателем, основанным на производной первого порядка ($\sim$) вместо глобального степенного показателя, что приводит к $g = \frac{d \log C_{\setminus E}}{d \log N_{\setminus E}}$. Смотрите полную информацию о том, как экспонента g приблизительно приведено в Приложении A.1 в [Pearce & Song \(2024\)](#).

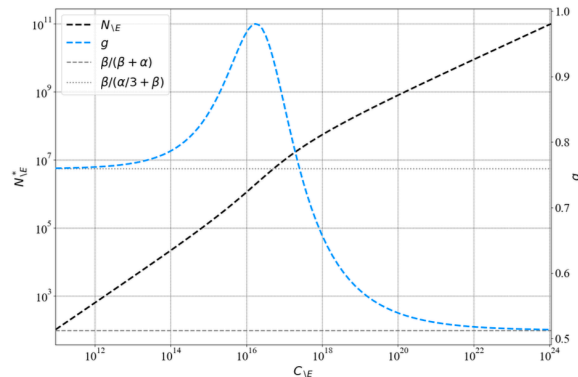


Figure 12: Visualization of how the local power-law exponent g grows with $C_{\setminus E}$.
(Image source: Pearce & Song 2024)

As shown in the visualization above, as $C_{\setminus E}$ gets larger, g converges to the Chinchilla estimate. By generating synthetic training curves using above equation, in the range of model size from 768M to 1.5B (as in Kaplan et al.), they estimated that g is close to the Kaplan coefficient of 0.73 in that region.

Why power law?

Power laws are widely observed across many domains outside AI, such as in [Zipf's law](#), [scale-free networks](#), [urban scaling laws](#), and many other complex systems. The recurring pattern is that large events are rare, small events are common and the relationship between size and frequency often follows a straight line at log-log scale.

Why do LLM scaling laws also have the shape of a power law?

Inspired partly by different domains displaying different exponents ([Hestness et al. 2017](#)), one early explanation by [Sharma & Kaplan \(2020\)](#) hypothesizes that language modeling can be viewed as doing regression on a low-dimensional manifold of data. More model parameters can induce a finer partition of the data manifold and therefore smaller generalization error. In the simplest terms, if a model of effective size N partitions a d -dimensional manifold into $O(N)$ regions, the typical linear resolution scales like $\sim N^{-1/d}$. This has a similar power-law form to the scaling laws above. This theory applies most cleanly in the infinite-data, underfitting regime, but in reality estimating the intrinsic dimension of a data manifold is quite hard.

A later hypothesis ([Michaud et al. 2023](#), [Brill 2024](#)) assumes that knowledge or skills are learned in discrete chunks ("quantized") and that the frequency distribution of these skills follows a power law. The model learns common skills first and rare skills later, resulting in a smooth power-law decay in loss.

I only listed two hypotheses here, but there are more studies on explaining the shape of power-law scaling through spectral tails of data, kernel eigenvalues, natural-language statistics, or phase transitions in training dynamics.

Scaling Laws in Data-Limited Region

Classic scaling laws assume effectively *unlimited unique data*, no repetition, and no multi-epoch training. As the model size grows significantly, we are running out of enough high-quality unique tokens. In fact, some arguments about how long scaling in AI can continue are centered on whether we are hitting a "data wall".

It is also worth emphasizing that the dataset behind D is expected to be already cleaned. The pretraining data pipeline is often a large part of an effective pretraining pipeline, with common steps like deduplication (exact and fuzzy), quality filtering, boilerplate removal, safety filtering, PII/copyright masking, benchmark decontamination and careful reweighting of data mix components based on language, quality, content type, etc. Even when two datasets contain the same token count D , a high-quality dataset and a dataset of Internet slop can yield drastically different compute efficiency.

The study by [Hernandez et al. \(2022\)](#) focused on a controlled version: a mostly-unique dataset with a small fraction of repeated data. Starting from a large dataset, the data mix keeps 90% non-repeated but replaces the remaining 10% with repeats of a tiny portion of the original. By training a Transformer model for 100B tokens, they observed a double-descent phenomenon, that is, the test loss can actually get worse and then better again as a function of how much the repeated data is emphasized, an effect that becomes more pronounced as the repeated fraction grows.

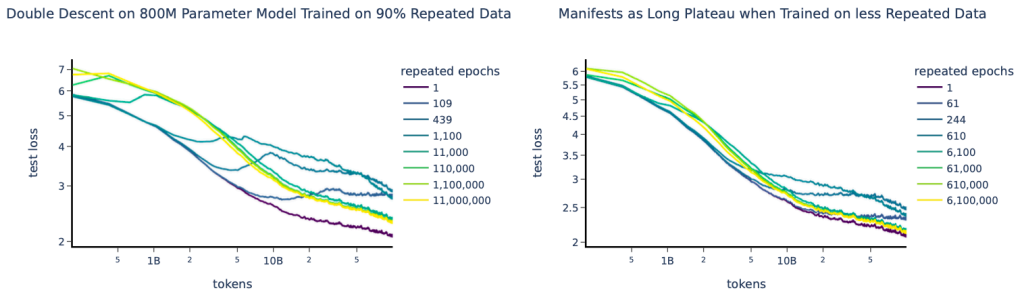


Figure 13: Double-descent in the test loss as the repeated fraction increases (90% repeated on the left, 50% on the right). (Image source: Hernandez et al. 2022)

The flat or increasing trend in the middle of training is possibly due to memorization of repeated data. Learning curves with such shapes make scaling law fitting less accurate. They also concluded repeated data hurts some OOD evaluation and downstream fine-tuning. However, their data mix is constructed in a more lab-like setup, and repetition in real-world data is often more nuanced (e.g. different data has different levels of repetition, semantic repetition, etc.).

Rather than saying data repetition hurts training, we are more interested in how to fit scaling laws, given that the unique high-quality data is not infinite and we likely have to repeat data during training.

[Muennighoff et al. \(2023\)](#) took on the research question of how compute should be allocated optimally when model training is data-constrained. Specifically, they empirically studied the impact of data repetition across roughly 400 experiments, 10M–9B parameters, data sizes up to 900B tokens, and up to 1500 epochs. The exact same dataset is repeated each epoch, shuffled between epochs, and evaluated on a held-out test set.

The key modeling adjustment is to decompose the total token count D into two parts: (i) the number of unique tokens U_D and (ii) the number of repeats R_D (i.e. num. epochs - 1). Thus we have $D = U_D(1 + R_D)$. With a unique-data budget D_{uniq} , by definition $U_D = \min\{D_{\text{uniq}}, D\}$ and $R_D = (D/U_D) - 1$. They use the Chinchilla scaling laws to find the optimal model size U_N for fitting U_D , and define excess model size via repeats $R_N = (N/U_N) - 1$.

They then update the Chinchilla parametric fit ([method 3](#)) to use effective (discounted) data D' and model size N' in place of the raw quantities:

$$\hat{L}(N, D) = \frac{A}{N'^{\alpha}} + \frac{B}{D'^{\beta}} + E \quad \text{where } D' = U_D + U_D r_D \left(1 - \exp\left(-\frac{R_D}{r_D}\right)\right).$$

The intuition is that a token's value decays *exponentially* as it is repeated. In their modeling, each repetition costs the token a $(1 - 1/r_D)$ fraction of its remaining value, where r_D is a learnable “half-life” parameter. When $R_D = 0$ or $R_D \ll r_D$, we recover $D' \approx D$.

A symmetric formulation handles excess model size, $N' = U_N + U_N r_N (1 - \exp(-R_N/r_N))$, capturing the idea that “larger models overfit more quickly on repeated data” and that “a model can be

too large for its dataset.” This component is less intuitive, and I could not find a satisfactory explanation for why model size needs to appear in such a symmetric form as repeated data. Later work by [Lovelace et al. \(2026\)](#) changed this assumption.

Their empirical fit finds that *excess parameters decay faster in value than repeated data*, $r_N < r_D$, so we should allocate more resources on more epochs rather than more model parameters. One weakness of this modeling, as the authors also pointed out, is that it significantly underestimates the final test loss of failing models (i.e. models whose loss increases midway through training), such as models trained for 44 epochs.

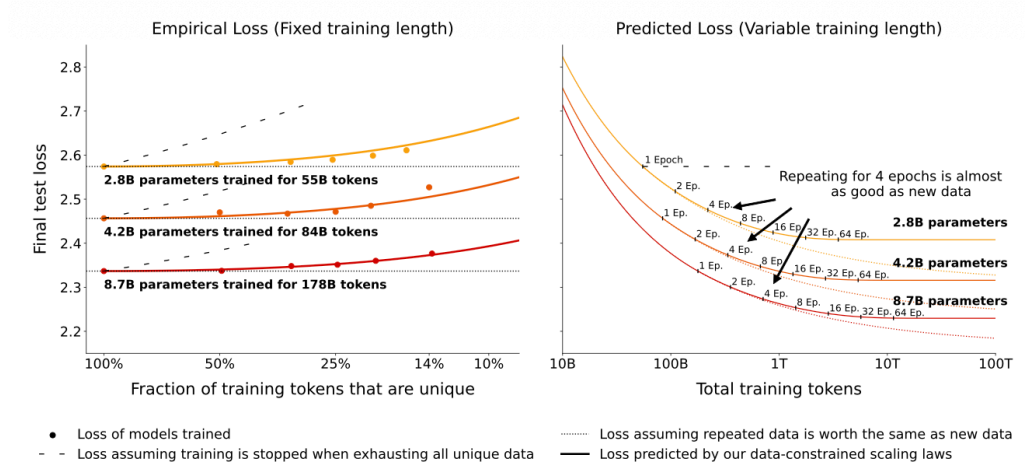


Figure 14: Data-constrained scaling under repetition captures the experimental results better than data-unaware fitting; the value of repeated tokens decays exponentially toward a ceiling. The fitting gets worse with more epochs as high repetition causes the test loss to increase midway through training, not depicted in the plot. (Image source: Muennighoff et al. 2023)

Most recently, [Lovelace et al. \(2026\)](#) revisited the same problem with a different approach. Rather than modeling overparameterization as a diminishing return on effective model size, Lovelace et al. model the interaction between model size $\times$ data repetition explicitly. Empirically, they trained about 300 models, spanning 15M to 1B parameters and 50M to 6B unique tokens.

When they plot the fit residual for a fixed model size across a range of data-repetition levels, the observation is intuitive: more epochs cause more damage, and interestingly *larger models are more sensitive* to repetition. This hints that the loss penalty is likely a function of both model size and data size.

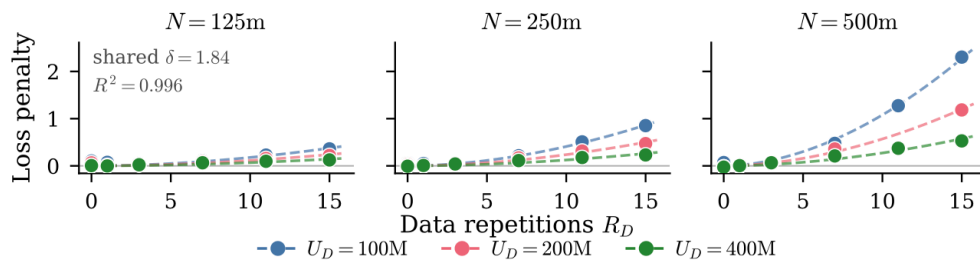


Figure 15: Residuals of the effective-size fit reveal that overfitting damage grows with both the number of epochs and the model size. (Image source: Lovelace et al. 2026)

An explicit overfitting penalty term was introduced and built around the *capacity ratio* N/U_D (parameter count relative to unique tokens):

$$\hat{L}(N, U_D, R_D) = E + \frac{A}{N^\alpha} + \frac{B}{(U_D(1 + R_D))^\beta} + P \cdot R_D^\delta \cdot \left(\frac{N}{U_D}\right)^\kappa$$

where:

- R_D is the repetition count;
- the scalar P is a learnable parameter;
- the exponent κ (the 2nd learnable parameter) lets the penalty scale nonlinearly with the capacity ratio N/U_D ;
- the separate exponent δ (the 3rd learnable parameter) on the repetition count decouples repetition nonlinearity from κ .

The added term (in red) is a direct overfitting penalty that grows with both how many times you repeat the data and how over-parameterized the model is relative to the unique data available.

They also did a case study on how weight decay impacts training with the limited-data constraint and found that strong weight decay reduces the overfitting penalty caused by data repetition.

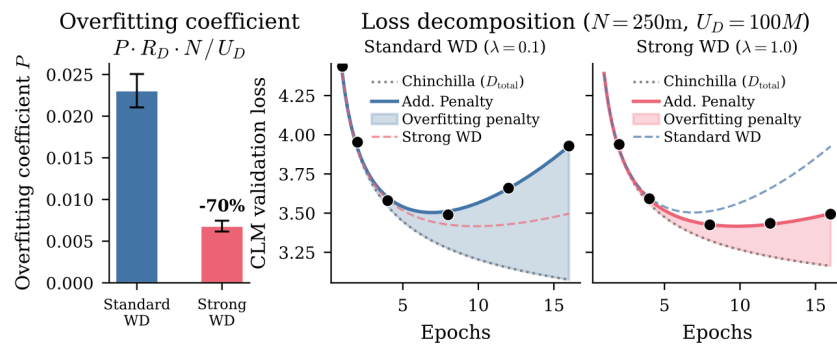


Figure 16: Strong weight decay reduces the overfitting penalty from data repetition. (Image source: Lovelace et al. 2026)

Both modeling approaches by Muennighoff et al. and Lovelace et al. are constructed from empirical curve fitting, so it is still unclear why data-constrained scaling laws should have exactly these forms and why each free parameter is needed. Curious about more theoretical work along this line.

Trickiness of Fitting Scaling Laws in Reality

Despite its clean form, in practice, scaling law fitting can be surprisingly sensitive to seemingly trivial procedural choices, like how you count parameters, how you round the precision, how you sum or average the loss, etc.

Because a scaling law is only fit on the (relatively small, relatively cheap) models that we can afford to train, and the prediction is *extrapolated* for a model orders of magnitude larger. In such a setup, choices that look like rounding error may lead to wild differences in prediction.

Meanwhile, scaling-law fitting assumes the only changing factor is *scale*, which means that the model architecture, optimizer, learning rate schedule, batch ramp, data mix, tokenizer, and other design choices should remain the same. Another underlying assumption is that all these settings should have been carefully tuned, as cases like undertrained models can lead to a different conclusion.

The disagreement between results by Kaplan et al. and Chinchilla is one example to showcase the trickiness of scaling laws fitting.

A second example is a follow-up analysis investigating why Chinchilla [method 3](#) is slightly off from the other two methods. [Besiroglu et al. \(2024\)](#) extracted the raw (N, D, L) data points from Figure 4 of Hoffmann et al. (2022) and re-ran the method 3 parametric fitting. They found a couple of concrete issues:

- A high loss scale in the L-BFGS-B minimizer, caused by averaging Huber-loss values over examples instead of summing them, which led to premature termination of the optimization. The early stopping of loss minimization during both the original fit and bootstrapping produced inconsistent estimates and implausibly narrow confidence intervals.

- The reported α and β were rounded to 2 digits of precision, which made the derived A, B look more off than they really were.

Toy simulation

Here is a toy simulation widget, created by ChatGPT, designed to demonstrate three specific failure modes.

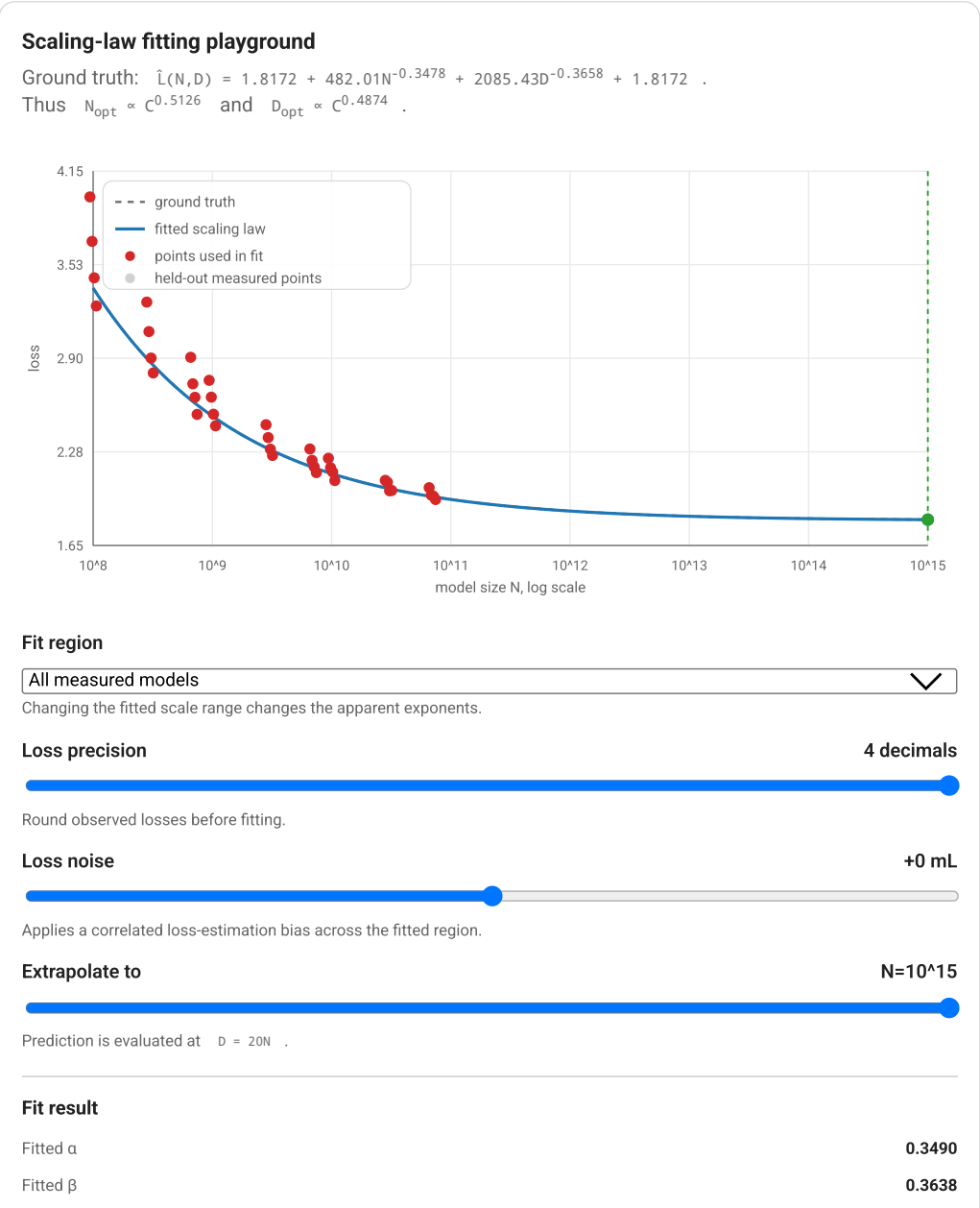
We assume the ground truth function is:

$$\hat{L}(N, D) = 482.01 \cdot N^{-0.3478} + 2085.43D^{-0.3658} + 1.8172$$

and thus $N_{\text{opt}} \propto C^{0.5126}$, $D_{\text{opt}} \propto C^{0.4874}$. This is the estimate from [Besiroglu et al. \(2024\)](#).

The simulation plots the loss prediction $\hat{L}$ vs dataset size D , while providing a set of sliders to show case:

- Loss precision: rounding losses from high to low decimal points can change the fitted parameter values.
- Loss noise: perturbing loss values by only a multiplier of milli-loss (0.001) units leads to different fit.
- Fit-region sensitivity: fitting only small models, only medium models, or all models gives different apparent scaling laws.



Fitted E	1.8167
Fitted N _{opt} exp.	0.5104 / true 0.5126
Fitted D _{opt} exp.	0.4896 / true 0.4874
Predicted loss	1.8224
True loss	1.8229
Error	-0.0005

Red points are used in the fit; gray points are held out. Points at the same model size use different token budgets. The dashed gray curve is the synthetic ground truth..

Citation

Please cite this work as:

Weng, Lilian. "Scaling Laws, Carefully". Lil'Log (Jun 2026).
<https://lilianweng.github.io/posts/2026-06-24-scaling-laws/>

Or use the BibTex citation:

```
@article{weng2026scaling,  
  title = {Scaling Laws, Carefully},  
  author = {Weng, Lilian},  
  journal = {lilianweng.github.io},  
  year = {2026},  
  month = {June},  
  url = "https://lilianweng.github.io/posts/2026-06-24-scaling-laws/"  
}
```

References

[1] S. Amari, N. Fujita, and S. Shinomoto. "Four Types of Learning Curves. Neural Computation." 4(4):605–618, 1992.

[2] Hestness et al. "Deep Learning Scaling is Predictable, Empirically." arXiv preprint arXiv:1712.00409, 2017.

[3] Rosenfeld et al. "A Constructive Prediction of the Generalization Error Across Scales." ICLR 2020.

[4] Kaplan et al. "Scaling Laws for Neural Language Models." arXiv preprint arXiv:2001.08361, 2020.

[5] Hoffmann et al. "Training Compute-Optimal Large Language Models." NeurIPS 2022.

[6] Pearce and Song. "Reconciling Kaplan and Chinchilla Scaling Laws." TMLR 2024.

[7] Bahri et al. "Explaining Neural Scaling Laws." arXiv preprint arXiv:2102.06701, 2021.

[8] Sharma and Kaplan. "A Neural Scaling Law from the Dimension of the Data Manifold." arXiv preprint arXiv:2004.10802, 2020.

[9] Hernandez et al. "Scaling Laws and Interpretability of Learning from Repeated Data." arXiv preprint arXiv:2205.10487, 2022.

[10] Muennighoff et al. "Scaling Data-Constrained Language Models." NeurIPS 2023.

[11] Lovelace et al. "Prescriptive Scaling Laws for Data Constrained Training." arXiv preprint arXiv:2605.01640, 2026.

[12] Besiroglu et al. "Chinchilla Scaling: A Replication Attempt." arXiv preprint arXiv:2404.10102, 2024.

[13] Michaud et al. "The Quantization Model of Neural Scaling" NeurIPS 2023.



[14] Brill. ["Neural Scaling Laws Rooted in the Data Distribution."](#) arXiv preprint arXiv:2412.07942, 2024.

[15] Rae et al. ["Scaling Language Models: Methods, Analysis & Insights from Training Gopher."](#) arXiv preprint arXiv:2112.11446, 2021.

Language-Model

Foundation

Pretraining

Transformer

Learning-Dynamics

«

Harness Engineering for Self-Improvement

»

Why We Think



